

Ephemeral Rust WebAssembly P2P Chat

<https://gemini.google.com/app/153ce58a33993b95>

User prompt: I would like to make an disposable chat app in rust webassembly that do not use emscripten. This app should support chat and audio calls (and video in the future). The idea is that an url provide the app and is stored in cache but the chat is only stored in ram. If the user refreshes the page it will lose everything about the chat. The chat communication should be p2p and secure.

Response: Building a disposable, browser-cached P2P chat app in pure Rust targets the wasm32-unknown-unknown target, bypassing Emscripten entirely. WebRTC via web-sys handles end-to-end encrypted audio and text, while browser linear memory guarantees complete ephemerality upon page refresh.

Core Tech Stack

- **Frontend Framework:** Leptos or Yew compiled to wasm32-unknown-unknown.
- **P2P & Audio Engine:** web-sys crate with bindings enabled for RtcPeerConnection, RtcDataChannel, MediaStream, and AudioTrack.
- **Signaling Relay:** A minimal, lightweight WebSocket server (e.g., Axum) solely used to swap WebRTC SDP offers/answers and ICE candidates.
- **Caching Layer:** Service Worker to cache static .wasm, .js, and HTML assets for instant app loading without storing user state.

Cargo Setup (Cargo.toml)

```
[dependencies]
wasm-bindgen = "0.2"
gloo-net = "0.6" # WebSocket signaling
leptos = { version = "0.6", features = ["csr"] }

[dependencies.web-sys]
version = "0.3"
features = [
    "RtcPeerConnection", "RtcDataChannel", "RtcConfiguration",
    "RtcSdpType", "RtcSessionDescription", "RtcSessionDescriptionInit",
    "RtcIceCandidate", "MediaStream", "MediaDevices", "Navigator", "MediaStreamConstraints"
]
```

Ephemerality & Security Strategy

- **Zero Persistence Storage:** Avoid localStorage, sessionStorage, and IndexedDB. All state (chat history, peer IDs, encryption keys) remains inside Rust struct fields or signals residing strictly in Wasm linear memory.
- **Native E2EE Encryption:** WebRTC mandates DTLS for RTCDataChannel (text) and DTLS-SRTP for audio/video media streams. Peer-to-peer traffic is encrypted out-of-the-box.
- **Instant Destruction:** Closing the tab or reloading destroys the Wasm instance and garbage-collects linear memory, permanently erasing state.

Minimal WebRTC Peer_INITIALIZER

```
use web_sys::{RtcConfiguration, RtcPeerConnection, RtcDataChannelInit};
use wasm_bindgen::prelude::*;

pub fn create_peer_connection() -> Result<(RtcPeerConnection, web_sys::RtcDataChannel), JsValue> {
    let mut config = RtcConfiguration::new();
    let ice_servers = js_sys::Array::new();
    let stun_server = js_sys::Object::new();
    js_sys::Reflect::set(&stun_server, &"urls".into(), &"stun:stun.l.google.com:19302".into())?;
    ice_servers.push(&stun_server);
    config.ice_servers(&ice_servers);

    let peer = RtcPeerConnection::new_with_configuration(&config)?;
    let mut dc_init = RtcDataChannelInit::new();
    dc_init.ordered(true);
    let data_channel = peer.create_data_channel_with_data_channel_dict("chat", &dc_init);
```

```
Ok((peer, data_channel))  
}
```

Would you like to focus next on implementing the WebSocket signaling exchange logic or configuring the WebRTC audio capture constraints?

User prompt: I would like to know how person x will connect to person y

Response: To establish a direct P2P WebRTC connection without persistent storage, Person X and Person Y rely on a temporary **Signaling Server** (a lightweight WebSocket relay) solely to exchange initial network metadata. Once connected, the server is completely bypassed.

1. Session Discovery

- **Person X** opens the Wasm app. The app generates a random, ephemeral room ID and appends it to the URL fragment (e.g., `mychat.app/#room-9x2f`).
- Person X shares this link with **Person Y** through an external channel (e.g., email or SMS).
- Both users' browsers connect to the WebSocket signaling server and subscribe to `room-9x2f`.

2. The SDP Offer (Person X)

- Person X's Wasm app creates an `RTCPeerConnection`, hooks up the audio stream and text data channel, and generates an **SDP Offer** (a document describing X's audio codecs, encryption parameters, and capabilities).
- Person X sets this as their `LocalDescription` and sends the offer to Person Y via the WebSocket relay.

3. The SDP Answer (Person Y)

- Person Y receives the offer, sets it as their `RemoteDescription`, and generates an **SDP Answer**.
- Person Y sets the answer as their `LocalDescription` and sends it back to Person X via the WebSocket. Person X sets this answer as their `RemoteDescription`.

4. ICE Candidate Trickling

- Simultaneously, both browsers query public **STUN servers** (e.g., Google's public STUN) to discover their own public-facing IP addresses, NAT types, and UDP ports.
- As these network paths (**ICE Candidates**) are discovered, the Wasm apps send them to each other through the WebSocket relay until a viable network path is found.

5. P2P Lock & Server Disconnect

- Once a mutual ICE candidate pair succeeds, the browsers negotiate a direct, encrypted DTLS/SRTP connection.
- **The WebSocket connection closes.**
- Audio and text now flow directly from Person X to Person Y across the internet, completely bypassing any intermediary servers.

If either person refreshes their browser, the Wasm memory wipes, the peer connection drops, and the session vanishes forever.
